Page No
Date

—— Mid sem End ——

Views

Base Table : Students (stored) → Virtual Projection → View : Tp of query (Not stored)

Uses
- → Granular Permission access b/w users
- → Support and transition from one system (legacy) to another (new)

CREATE VIEW viewname AS (select - - - - - - - - - -);
↳ Any query that returns a relation

→ Modifying base modifies View also

→ Querying is same as that of Table

→ Delete ⇒ DROP VIEW viewname;

Updatable Views

→ Allows DML - INSERT, UPDATE, DELETE - And these are passed through to base ~~table~~ table

⇒ Updatable if
- → view creating SELECT does not contain DISTINCT
- → Query is based on a single table
- → Query shouldn't use aggregate functions
- → No group by or having
- → No subqueries

→ Creation is same

→ UPDATE viewname

Materialized Views

⇒ Why?
- → Since regular views are virtual, there is a performance overhead
- → Real time update is not important

→ Materialized Views are physically stored

→ CREATE MATERIALIZED VIEW Vname as ( )
query which returns table

→ No connection/relⁿ b/w base and view

→ The Stale Data Problem: Maintenance of Materialized Views
↳ REFRESH MATERIALIZED VIEW Vname;
→ Can be manual
→ Can be scheduled → Automatic (requires some Trigger)

Indexes

→ Table Scan: slow
→ Index → hash table → faster lookup

→ CREATE INDEX IDname ON Tname (colname);
↳ order of colname matters

→ CREATE UNIQUE INDEX ——— ——— ———
↳ Only if indexed col has unique values
↳ P Key is automatically a unique index

→ Tradeoff

Search speed v/s Modification speed

Pros

→ faster, select, Join, order by

Cons

→ slower insert, update, delete

→ Indexes take space

→ Poorly chosen indices can slow down

Decision pts.

Which columns to idx

→ Frequently used in where and order by

→ Foreign key for Join

Date

→ Storage Hierarchy

→ Storage Media Types

Primary — Main Memory
Secondary — Magnetic Disk HDD
Anatomy of disk read/write
Disk Scheduling — Elevator (SCAN) Algorithm
Data Placement Optimization
Secondary — SSD
Write Endurance
Wear Levelling

→ File System Organization

Blocks and Pages
Making faster Access
Pointer Swizzling
Storing Records in Files
Fixed sized records
Handling Insertion and Deletions
Variable length records
Byte string representation
Slotted Page Structure

hex-mode

$2^3 \times 2^{12}$ 2^{15} $5 \times 2^8 + 15 \times 2^4 + 1$ 85f1 $8 \times 16^3 + 5 \times 256 + 15 \times 16 + 1$

8192 / 4 / 2048 / 1024 / 512 / 256

Page No
Date

Organization of Records in File

Heap, Sequential, Hashing

Any record can go to any page

Best for Bulk loading

Ordered on the basis of search key

Best for Many queries less updates

hash fn on search key

Best for: very fast access based on exact key values

Performance on different operations

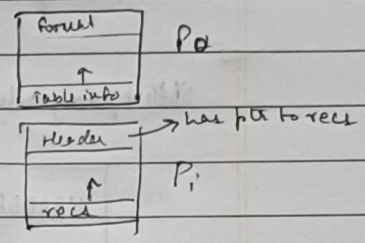
$4096 + 15 \times 256 + 15 \times (16) + 15 \times 1$

$4096 + 15(273)$

$4096 + 4095$

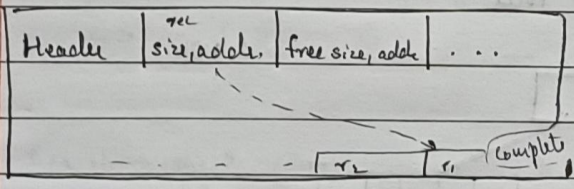
8191 b

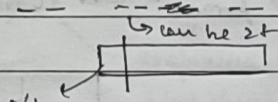
273 / 15 / 135 / 2730 / 4095



if key present then key = rowid else assigned by sqlite3

length rowid num-cols meta stores





0/1

next byte is also a part of curr field

1113 / 16384 / 4096 / 8 / 21282

0 0 0 0 0 1 0 1 1 1 1 0 0 0 1 0 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 1 0

23 22 21 20 19 18 12 6 5

16384 4096 512 256 32 2

4888 802 240 34

$2^{23} + 2^{21} + 2^{20} + 2^{19} + 2^{18} +$

$2^{18}(2^5 + 2^3 + 2^2 + 2 + 1) + 21282$

32 8 4 2 1

$47 \times 2^{18} + 21282$

~~varchar-meta — 12 bytes~~

varchar-meta − 13 / 2 = len

Metadata and Buffering

Metadata

Buffering

B. Hit

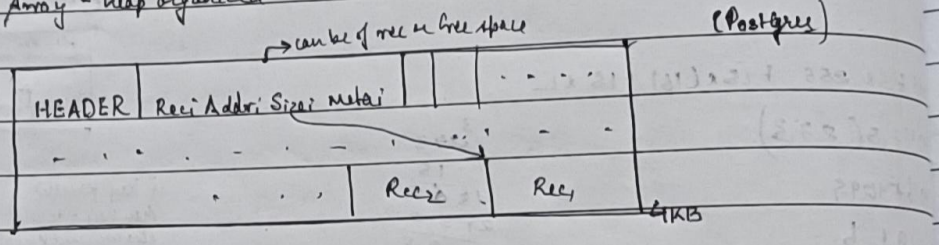
B. Miss

B. manager

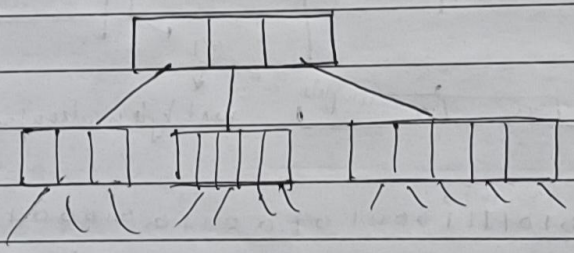
replacement policy

B. Diagram

Slotted Array - Heap organized



Slotted Array - Index Organized Files



Page str. is similar.

Disadv.

↳ Fragmentation → Deletion leaves blank space ⇒ variable sized ⇒ fragmentation

↳ Updation → No room to grow ⇒ Effectively delete + insert new

↳ Useless I/O → Entire pages must be fetched to find a record

↳ Random disk I/O → Random back and forth ⇒ Operations may not be sequential

Page No
Date

Log Structured Storage

Two operations for write – PUT, DELETE

<rowid> t_1 <rowid>

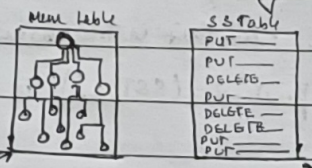
key which a tuple

→ sequential

	row	value (s)
PUT	r_1	t_1
PUT	r_2	t_2
PUT	r_3	t_3
PUT	r_4	t_4
DELETE	r_3	t_3
PUT	r_2	t_2'

Search direction

→ Improving search → MemTable for main memory and update



Update this directly

→ compressed logs (only a few operations (only last row))

Search → check memtable

found

not found → SSTable

↳ Pushed to Disk

OLD → NEW

→ Storage in HDD

↳ Try to keep in log structure

↳ level wise compaction

L_0 → ss table from mem

L_1 → compaction of multi tables

L_2

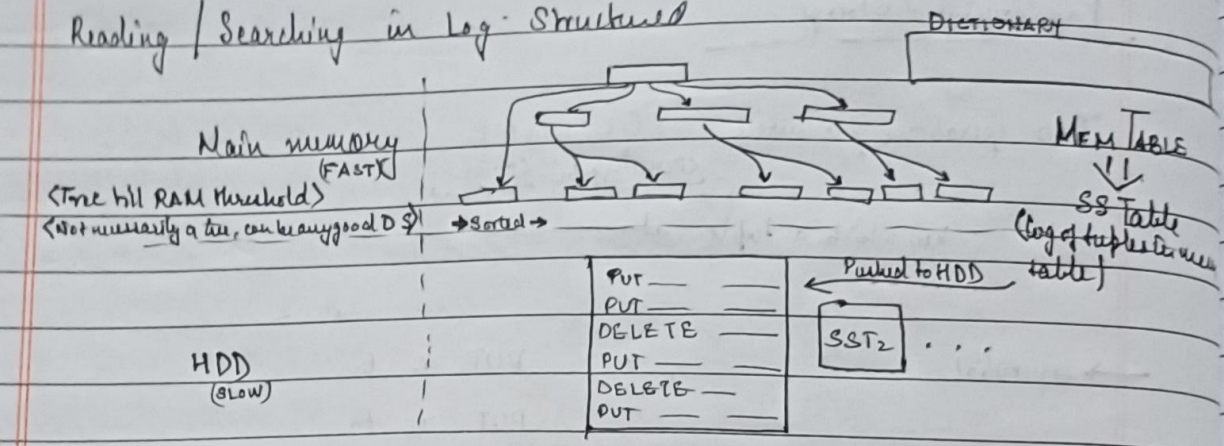
L_3

once done, delete from L_0

For B Tree, the leaf nodes (tuples) are in sorted order

SS Table → Sorted Structured Table

Reading / Searching in Log-Structured



Deleted data should also be pushed in mem table (Foreg: 102, NULL delete marker) to have proper track of operations for SS Table

After pushing SST to HDD, Mem table in RAM is deleted making space for another mem table. All this is time-bound / sequential (?)

The last SST pushed (SST_{max}) is the latest / newest record.

Search order: Mem Table → SST_n → SST_{n-1} → ... → SST_2 → SST_1

SST_i reading — Find SST_i in HDD → Push to RAM → Read SST_i

→ This is still slow — so we can't search from n to 1

→ To improve this, we store a DICTIONARY (any good DS) storing key ranges stored in each SST

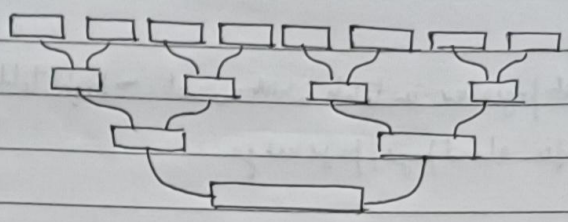
→ Still in worst case, we would have to search all

→ (improvement) COMPACTION → b/w 2 (or more) tables, pick latest record for each record/row

→ since logs/SST is in sorted order, this becomes simpler for B+ Trees.

Page No
Date

We make levels of compaction



Once higher lvl is created, we can delete old SSTs
Also, our implementation technique is to make the intervals mutually exclusive

Alternative → Filter (can be probabilistic - Bloom filters)

New search order : Mem Table → Filter/Dict → l_0 → l_1 → l_2 → . . . .
given ↑ table for each lvl

⇒ Read-Friendly, Write/Insertion ~~can be troublesome due to compaction overhead~~ is decent (?)

Buffer

⇒ Storage Control and Buffer Pool

— Need for storage control
  ↳ spatial control : where on disk → to reduce fwin disk arm movement
  ↳ Temporal control : When to move data ↔ disk → to avoid slow disk I/O → job of Buffer Manager

— Role of Buffer Pool
→ region of main memory that caches disk pages
→ same like OS

Page No
Date

Buffer Pool Organization

→ Frames → Pages
→ Page Table → which pages are in table, which are not → hash table
→ Metadata → info about page / page usage

Metadata and Latches:
→ Dirty Bit
→ Pin count → How many ops are currently using the page
→

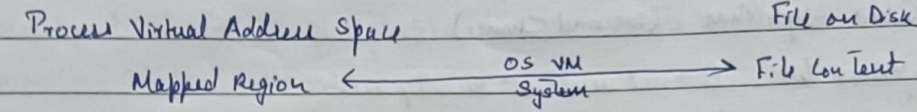
Thread safe Page table → using (mutex and semaphores)
↳ such locks (mutexes) are known as latches

→ In dbs → locks and latches are different
for db contents ↳ ↳ for low level internal
high level memory structures

Memory Mapped I/O (mmap)
→ Virtual mem.

Page No
Date

⇒ Memory mapped - I/O
(mmap() system call)



→ OS may have larger page sizes for DBMS
→ BUT, OS does not know the eviction policy and buffer management policy specific to DBMS, it uses the one for general workload.

→ To improve / counter that, OS provides system calls for re-map regions
  ↳ madvise(): hint OS about how you expect to use the mem region
  ↳ mlock(): Pin it to mem, prevent swap to disk
  ↳ msync(): Flush back to disk

PROBLEMS
→ Transaction safety → OS swaps out when data is being written / modified
→ I/O stalls →
→ Performance issues → DBMS have a better idea about what can be next, OS will follow general policy evict

=

⇒ Buffer Replacement Policy

Goal: Evict page which is ~~to~~ least likely ~~not~~ to be needed

Imp Things	Key Policies
- Correctness	- LRU
- Accuracy	- Clock
- Speed	- LRU-K
- Meta-data overhead	- ARC (Adaptive Replacement Cache)

→ LRU

→ least recently used

→ Clock Algorithm (Second Chance)

→ reference bit 0 → not used 1 → recently used
evict zero ref bit pages

Problem → Sequential Flooding

→ LRU and clock perform poorly

→ A large scan can fill the buffer with pages that will only be used once → evicts all other frequently used pages to accomodate new pages

→ say after putting all windows by window, we want to select top k → this would also require eviction

→ LRU-K

→ considers more than the last access time

→ keeps track of last k access timestamp

→ victim will have largest backward k distance (k^th most recent acc)

→ pages accessed <K times will have ∞ distance → prime eviction candidate

(MySQL uses modified (approx.) LRU-K)

→ Other Policies

→ ARC = Adaptive Replacement Cache

- maintains 2 LRU lists → recently accessed pages T_1
  → frequently accessed pages T_2
- adaptively changes size based on load

→ GHOST ≈ PAGES → previous access times are kept

→ Localization (per-process buffer) → assign portion of buffer pool for specific query to ensure un-interference

→ Priority Hints :- ▷ query optimizer hints buffer manager about important pages to not evict them

OS Interactions

→ Taking care of dirty pages :- Write Ahead Logging (WAL) to ensure durability.

→ DBMS -v- OS Disk Scheduling

→ Avoiding OS Page Cache

→ Prefetching and Scan sharing

B+ Tree

⟹ Indices on non-unique values (duplicate containing values)

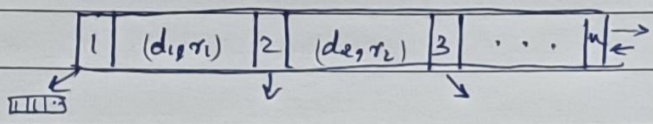
recommended → attach rowid (record id) to ~~each~~ table → contains unique values

for each tuple → primary key can be used as rowid

→ some still create a rowid even if primary key is present.

→ B^+ tree created on (dupe-col, rowid) (similar to composite key)

(eg: (discipline, rowid))



<Not the case in heap org-storage>

In sqlite 3 → If index is on primary key → storage will be sequential

Clustered Index → re-sorts pages → Based on a column → ~~often~~ not a living de

In SQLite3 → Index Organized Form → for storing pages

↳ Index on rowid / 1° key

This sequential behavior is good for HDD as back-forward movement is reduced

clustered index gives an option for heap orgs to sequence the record but it can again get messy after more insertion-deletions

---

~~Reco~~ Accessing Pages

→ Keep list of pages to be accessed instead of picking one-by-one along with the offsets

→ cluster / club same pages together

→ read cluster by cluster

→ unless specified, recs are shown in order of insertion (due to order inside the leaf nodes)

→ for complex -2+ col queries, dbms figures out intersection of bits then reads clusters.

Page No
Date

Optimizations

↳ sibling ptrs.

↳ size of node: 1 pagesize (depends on type of hard disk used)

↳ tradeoff b/w searching within node and finding the node

↳ searching within the node: binary search

↳ merge-split conditions: merging can be delayed (can be unbalanced sometimes)

↳ expecting more insertions to balance the tree